CHAPTER 21

Metrics Monitoring and Alerting System

Collect, store, query, visualise and alert on time-series metrics from large infrastructure.

Jump to: Problem statement | Clarifying questions to ask | Requirements | Back-of-the-envelope estimation | API / data model | High-level design | Deep dive | Trade-offs and alternatives | Scaling, reliability, failure modes | Likely follow-up questions | Common mistakes | Interview answer script | 60-second recap / practice checklist | References and further reading

1. Problem statement

Design an internal infrastructure **metrics monitoring and alerting platform** (Prometheus / Datadog / Grafana style) for a large company: collect operational metrics (CPU, memory, request counts, error rates), store them as time series, visualise dashboards, and alert when thresholds are crossed.

Why interviewers ask it: A write-heavy time-series/streaming architecture with pull vs push collection, TSDB, downsampling, and alert evaluation.

2. Clarifying questions to ask (and sensible assumptions)

Question	Assumed answer
Who uses it?	Internal: infrastructure and service metrics only (not business analytics).
Scale?	100 M DAU, 1,000 server pools x 100 machines, 100 metrics per machine.
Retention?	1 year; raw 7 days, 1-minute resolution 30 days, 1-hour resolution 1 year.
Alert channels?	Email, PagerDuty, webhook.
Logs / tracing?	Out of scope (metrics only).

3. Requirements

Functional

- Collect metrics from many sources.
- Store and query time series.
- Dashboards.
- Alerting with notification channels.

Non-functional

- Scalable ingestion and queries.
- Low query latency.
- Reliable alerts (the monitoring system must not be down when others are).
- Flexible, extensible.

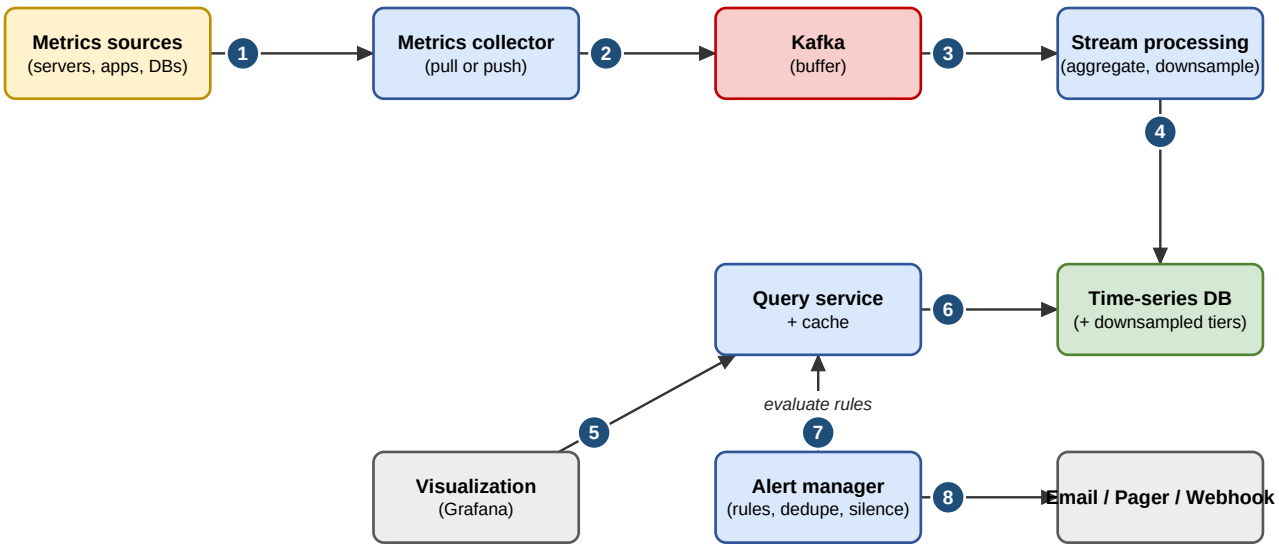
4. Back-of-the-envelope estimation

- 1,000 pools x 100 machines x 100 metrics = **10 M metrics**; with 10 s resolution -> about 1 M data points/s.
- Each point about 16 bytes (timestamp + float) -> 16 MB/s raw, about 1.4 TB/day before compression (TSDB delta encoding compresses 10x+).
- Reads: dashboards are periodic and alert rules evaluated each minute - far fewer than writes.

5. API / data model

```
Data model (line protocol):  metric_name{label1=v1,label2=v2}  timestamp  value
  cpu.load{host=web01,region=us-west}  1613707265  0.29
Query (PromQL-like): avg(rate(http_requests_total{service="api"}[5m])) by (region)
```

6. High-level design



Step	What happens
1	Hosts, services and databases expose metrics (CPU, requests, errors). Collectors **pull** them from `/metrics` endpoints (found via service discovery) or receive **pushes** from agents.
2	Collectors write the points to **Kafka**, which buffers bursts and decouples collection from storage.
3	**Stream processing** consumes, aggregates (rollups) and downsamples where useful.
4	Data is written to the **time-series DB** (compressed, label-indexed), with tiers: raw 7 d, 1-minute 30 d, 1-hour 1 y.
5	Dashboards (Grafana) send queries to the **query service**, which uses a query cache and picks the right resolution tier.
6	The query service reads series from the TSDB.
7	The **alert manager** evaluates alert rules against the query service every minute.
8	On a breach it dedupes, groups, applies silences and escalates to email, PagerDuty or webhook. Its state sits in a KV store to avoid flapping.

7. Deep dive

Pull vs push collection

- **Pull (Prometheus):** collector scrapes `/metrics` endpoints; needs service discovery (etcd/ZooKeeper/Consul) to know targets; easy to debug health; may struggle with short-lived jobs and firewalls.
- **Push (Datadog, CloudWatch):** agents on hosts push to the collector; good for short-lived jobs and behind-NAT; collector needs rate limiting/auth; aggregator on the agent to cut volume.
- Large setups often use both; scale collectors via consistent-hash ring so each collector owns a subset of targets.

Queue between collection and storage

- Kafka (partitioned by metric name/labels) decouples collectors from the TSDB, absorbs bursts, enables replay and multiple consumers (alerting, long-term storage). Trade-off: extra component; some systems (Prometheus) skip it for in-memory simplicity.

Time-series database

- Use a purpose-built TSDB (InfluxDB, Prometheus TSDB, OpenTSDB): handles heavy writes, compression (delta-of-delta, Gorilla), label indexing, and time-range queries efficiently; **do not** use a general RDBMS or plain NoSQL for this load.
- Data model: series identified by `name + sorted labels`; points are `(timestamp, value)`. **High cardinality** (e.g. user_id as a label) explodes series count - discourage it.

Aggregation, downsampling and retention

- Aggregate on the collection side (less data) or ingestion pipeline (stream processing) or at query time (flexible but slow).
- **Downsample** older data: raw 7 d -> 1-minute resolution for 30 d -> 1-hour for 1 y. Cold storage for oldest. Saves cost and keeps long-range queries fast.

Query service and visualisation

- A query API in front of the TSDB with a **query cache** for repeated dashboard queries; use the downsampled tier matching the time range. Visualisation with Grafana.

Alerting

- **Alert manager** periodically evaluates rules (YAML) against the query service; supports grouping, dedupe, silencing, escalation, and routing to channels; stores alert state in a KV store to avoid flapping and duplicates.
- Make the alerting path **independently reliable**: HA pairs, meta-monitoring (a watchdog alert confirming the pipeline works), multi-region.

Operability

- Monitor the monitoring: ingestion lag, queue depth, TSDB write latency, dropped points. Shard TSDB by metric name/time; replicate for HA.

8. Trade-offs and alternatives

Decision	Option A / Option B	How to choose
Collection	Pull / Push	Pull for observability & health insight; push for ephemeral/NAT'd sources.
Storage	TSDB / general NoSQL	TSDB: compression and time queries; NoSQL only for tiny scale.
Aggregation location	Agent / stream / query time	Mix: agent+stream for known rollups, query-time for ad hoc.
Buffering	Kafka / direct write	Kafka for resilience; direct for simplicity at small scale.

9. Scaling, reliability, failure modes

- Shard collectors and TSDB; Kafka partitions scale ingestion; tiered storage for cost.
- Rate limit and cardinality limits protect the system from noisy tenants.
- Multi-region with local ingestion and global query federation.

10. Likely follow-up questions

- How do you handle high-cardinality labels?
- How would you design anomaly detection or SLO burn-rate alerts?
- How do you ensure alerts still fire if your monitoring cluster is partially down?
- How would you federate multiple regional clusters?

11. Common mistakes

- Storing metrics in a relational DB.
- Not discussing downsampling/retention.
- Alert storms with no dedupe, grouping, or silencing.

12. Interview answer script (what to say, in order)

Follow the 4-step framework from chapter 04. Say it out loud; the quoted text is a model answer in your own voice.

Phase	Time	What to say
1. Understand + scope	5 min	"This is an internal infrastructure metrics platform, not business analytics. Assume 1,000 server pools of 100 machines each with 100 metrics, that is 10 M series and about 1 M data points per second at 10-second resolution. Retention: raw for 7 days, 1-minute for 30 days, 1-hour for a year. Alerts go to email, PagerDuty and webhooks."
2. High-level design	10 min	"Sources are scraped or pushed to collectors, buffered in Kafka, processed by a stream layer, and stored in a time-series database. A query service with a cache serves dashboards and the alert manager, which evaluates rules and notifies."
3. Deep dive	25 min	"Pull versus push: pull makes health easy to see and uses service discovery, push suits short-lived jobs; I'd support both. I use a TSDB because general databases can't handle the write rate or compression. Cardinality must be controlled. I downsample older data to save space and keep long-range queries fast. For alerting, the alert manager dedupes, groups and silences, and runs as an HA pair with a watchdog so I know if it dies."
4. Wrap up	4 min	"I'd monitor the monitoring system itself: ingestion lag, queue depth, dropped points. For global scale, regional clusters with a federated query layer. Extensions: anomaly detection and SLO burn-rate alerts."

13. 60-second recap / practice checklist

- Sources -> collectors (pull/push) -> Kafka -> stream processing -> TSDB -> query service -> dashboards and alert manager.
- Time-series specifics: labels, cardinality, compression, downsampling.
- Reliability of the alert path is non-negotiable.

14. References and further reading

Cross-checked against these sources (links verified when this PDF was built).

• Prometheus overview
• Gorilla: Facebook's in-memory TSDB
• Google SRE book: Monitoring distributed systems